

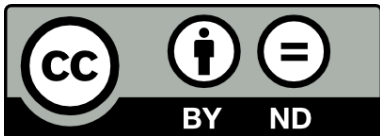
VPRM: A Reduced Order Model Framework
Applied to Viscoplasticity and High Temperature
Components

Billy Guy

25-12-22

Licence

This work is licensed under a Creative Commons Attribution-NoDerivatives 4.0 International License. You can view a copy of this license at <https://creativecommons.org/licenses/by-nd/4.0/>



Foreword

This book is a collection of knowledge, notes, and any learning material I had to use to learn or recall on topics like continuum mechanics, machine learning, nonlinear FEA and the use of open source softwares like libROM and MFEM. The end goal is to have a clean notebook with what I learned or what at I had to recall to implement my end goal: ***A Framework for Hyper-reduction of Viscoplastic Models applied to high temperature components***. While I took a lot of my free time (and cut on sleep once in a while), worked hard to test and verify to make the most out of this book it is most likely certain that it contains some error due to either formatting and/or comprehension. I used Large Language Models (LLMs) to answer questions or verify certain parts of the text, but still, I think some errors may have slipped in. It is not a sponsored project, this is something I worked on during my free time because I am genuinely interested by te topics adresses in this book. In no case this book should be taken as is. Each and every cited ressources should be read to fully understand the notes I took. I take no responsibility regarding the exactitude of any equations or any theory related claim. I would be happy to help someone learn, but the learner should help him/her self and take appropriate actions to validate his/her learning.

This project started because I wanted to learn and work on something relevant to the current finite element analysis (FEA) / reduced order models (ROM) / artificial intelligence (AI) landscapes. I think that AI's biggest problem related to simulation is data reliance. While technologies like Physics Informed Neural Networks (PINNs) are good specific machine learning techniques applied to simulation, other technologies still rely heavily on data to calibrate models. In this data driven world, I couldn't stand companies selling :*AI results in minutes rather than days!* when in fact, it took most likely days and numerous well thought and well built models to feed the "super fast" AI prediction. But thats when I had the idea to first evalutate the capability of a "non AI" solution to predict results of large scale, highly nonlinear problems. Maybe to even solve complex problem faster and with higher accuracy than an AI approach? Or maybe feed AI models more efficiently? Then I started reading on the topics and ask Claude (by Anthropic) to act as a supervisor and guide me through this quest to learn and find interestings things.

Billy Guy, M.Sc.A

Contents

Licence	i
Foreword	iii
1 Introduction	1
2 Elements of continuum mechanics	3
2.1 Tensor Fundamentals	3
2.1.1 Introduction	3
2.1.2 Matrices, Vectors and Tensors	4
2.1.3 Symbols and Tensor Operation	5
2.1.4 Problem set	6
2.2 Deformation gradient and motion	9
2.2.1 Introduction	9
2.2.2 Deformation Gradient	10
2.2.3 Polar Decomposition	13
2.2.4 Problem set	16
2.3 Motion	19

Chapter 1

Introduction

High temperature components are in a lot of critical applications and they are stressed in certain ways that common parts are not. The temperature effect induce certain material behavior that aren't really present in normal applications. One of these material behavior is viscoplasticity. Viscoplasticity is defined by the change in viscous response of the material due to the rate of solicitation.

WIP

Chapter 2

Elements of continuum mechanics

2.1 Tensor Fundamentals

Section References

The references used in this section are:

- Continuum Mechanics website [2]
 - Vectors
 - Matrices & Tensors
 - Tensor Notation (Basics)
 - Tensor Notation (Advanced)

This section will cover the vector and tensor essentials to understand tensor algebra applied to solid mechanics. Vectors and matrix are essentially containers for scalars. Scalars are ordered in a certain fashion and some operation can be done with them. Tensors are an extension to vector and matrix. In this report, this notation will be used:

- Scalars will be lower case letters, i.e.: $a, b, c, ..$
- Vectors will be lowercase bold letters, i.e.: $\mathbf{x}, \mathbf{v}, \mathbf{a}$ or $\mathbf{x}, \dot{\mathbf{x}}, \ddot{\mathbf{x}}$
- Matrix will be uppercase bold letters: $\mathbf{A}, \mathbf{C}, \mathbf{X}$

2.1.1 Introduction

This section is a basis for tensor definition and operation. It is not an in depth review of linear or tensor algebra but rather a simple, short take on the required

notions for continuum mechanics.

2.1.2 Matrices, Vectors and Tensors

A vector is defined as:

$$\mathbf{v} = \begin{pmatrix} i \\ \vdots \\ n \end{pmatrix}$$

A matrix is defined as:

$$\mathbf{M} = \begin{bmatrix} 1 & 2 & \dots & n \\ 2 & \ddots & & \vdots \\ \vdots & & \ddots & \vdots \\ m & \dots & \dots & mn \end{bmatrix}$$

The dot product of two vector is:

$$\mathbf{a} \cdot \mathbf{b} = a_i b_i = a_1 b_1 + a_2 b_2 + a_3 b_3 = c$$

Above, the example use tensor notation and c is a scalar. That means that the use of subscript defines the size of each container (vector = 1st order tensor) and how they interact with each other.

Key Concept 2.1.1: Einstein notation

There is one relatively simple rule with indices: if an index is repeated twice, it is summed. For example:

$$c_{ij} = a_i b_j \neq a_1 b_1 + a_2 b_2 + a_3 b_3$$

instead, $a_i b_j$ yields a matrix or a second order tensor:

$$c_{ij} = a_i b_j = \begin{bmatrix} a_1 b_1 & a_1 b_2 & a_1 b_3 \\ a_2 b_1 & a_2 b_2 & a_2 b_3 \\ a_3 b_1 & a_3 b_2 & a_3 b_3 \end{bmatrix}$$

As seen in the concept above, the way the indices are called defines how they are used. This introduces also the concept of tensor product and contraction. The first example $a_i b_i$ can be seen as the tensor contraction of two 1st order tensors:

$$\mathbf{a} : \mathbf{b} = a_i b_i = a_1 b_1 + a_2 b_2 + a_3 b_3$$

The later can be seen as the tensor product or dyadic product of two 1st order tensors:

$$\mathbf{a} \otimes \mathbf{b} = \begin{bmatrix} a_1 b_1 & a_1 b_2 & a_1 b_3 \\ a_2 b_1 & a_2 b_2 & a_2 b_3 \\ a_3 b_1 & a_3 b_2 & a_3 b_3 \end{bmatrix}$$

For the sake of comprehension, the same operations can be done with vector-matrix notation. Consiering both vectors $\mathbf{a}$ and $\mathbf{b}$ are column vectors :

$$\mathbf{a}^T \mathbf{b} = \mathbf{a} \cdot \mathbf{b} = \mathbf{a} : \mathbf{b}$$

and

$$\mathbf{a} \mathbf{b}^T = \mathbf{a} \otimes \mathbf{b}$$

Here, the difference between the tensor contraction and dot product is that tensor contraction works on higher tensor dimensions while the dot product only work on a combination of vector. The same goes for the dyadic product. An example of higher order tensor contraction would be the combination of the elastic stiffness tensor (4th order) and strain tensor (2nd order) that yields the stress tensor (2nd order):

$$\boldsymbol{\Sigma} = \mathbb{C} : \boldsymbol{E}$$

Important note: here, this expression could be reduced to a simpler expression using the symetries in $\mathbb{C}$ and using the difinition of stress and strain in Voigt notation, yielding:

$$\boldsymbol{\sigma} = \boldsymbol{C} : \boldsymbol{\varepsilon}$$

Voigt notation will be in [section n](#).

2.1.3 Symbols and Tensor Operation

Another use of index notation and helpful for expression contraction is the Kronecker Delta (δ_{ij}). The Kronecker Delta takes two inputs (i and j) and returns one if both are greater than zero (should always be) and are equal:

$$\delta_{ij} = \begin{cases} 1 & i=j, \\ 0 & \text{else} \end{cases}$$

Example 2.1.1: Identity matrix and the Kronecker Delta

The identity matrix in tensor notation is the Kronecker Delta.

$$\boldsymbol{I} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \delta_{ij}$$

Important note: the identity matrix is not the Kronecker Delta since it

is a scalar (either 0 or 1).

Another symbol called the permutation symbol (or the Levi-Civita symbol) is a scalar defined as $\epsilon_{ijk} \dots$. The $\dots$ means that it exists for higher indices the symbol and works with the same logic. The idea is that if it follows a certain ordering the return value is either 1, -1 or 0 :

$$\epsilon_{ijk} = \begin{cases} +1 & \text{if } (i, j, k) \text{ } (1, 2, 3), (2, 3, 1) \text{ or } (3, 1, 2) \rightarrow \text{even permutation ,} \\ -1 & \text{if } (i, j, k) \text{ } (3, 2, 1), (2, 1, 3) \text{ or } (1, 3, 2) \rightarrow \text{odd permutation ,} \\ 0 & \text{if any two indices are equal} \end{cases}$$

The Levi-Civita symbol can be used to define the cross product of two vector for example, let $\mathbf{a} \times \mathbf{b}$, thus:

$$c_i = \epsilon_{ijk} a_j b_k$$

Example 2.1.2: Cross product using Levi-Civita symbol ϵ_{ijk}

To find each component of the vector $\mathbf{c} = \mathbf{a} \times \mathbf{b}$, we take each component that are not equal to 0 in the permutation, thus:

$$\begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} \epsilon_{1jk} a_j b_k \\ \epsilon_{2jk} a_j b_k \\ \epsilon_{3jk} a_j b_k \end{bmatrix} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}$$

2.1.4 Problem set

Problem 2.1 *Einstein summation*

Given the following vectors:

$$\mathbf{a} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}$$

Find $a_i b_i$.

Answer: Since index i is repeated, we sum all components on i . Thus, it becomes a dot product. The result is:

```
a = np.array([2,-1,3])
b = np.array([1,4,-2])
c = np.dot(a,b)
```

$$\begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 4 \\ -2 \end{bmatrix} = -8$$

Problem 2.2 *Matrix-Vector product*

Given matrix A and vector x , compute $y_i = A_{ij}x_j$ for each component

Answer: Since index j is repeated, each y_i component will be a sum of combination of $A_{ij}b_j$.

```
from sympy import MatrixSymbol
A = MatrixSymbol('A', 3, 3)
x = MatrixSymbol('x', 3, 1)
y = MatrixSymbol('y', 3, 1)
y = A * x
```

$$A = \begin{bmatrix} A_{0,0} & A_{0,1} & A_{0,2} \\ A_{1,0} & A_{1,1} & A_{1,2} \\ A_{2,0} & A_{2,1} & A_{2,2} \end{bmatrix} x = \begin{bmatrix} x_{0,0} \\ x_{1,0} \\ x_{2,0} \end{bmatrix}$$

$$y = Ax$$

$$y = \begin{bmatrix} A_{0,0}x_{0,0} + A_{0,1}x_{1,0} + A_{0,2}x_{2,0} \\ A_{1,0}x_{0,0} + A_{1,1}x_{1,0} + A_{1,2}x_{2,0} \\ A_{2,0}x_{0,0} + A_{2,1}x_{1,0} + A_{2,2}x_{2,0} \end{bmatrix}$$

As expected, i is fixed on A and summed on j for both A and x . Note that Sympy indicate a double index notation for x even though that it is a vector. However, the second index doesn't change.

Problem 2.3 *Kronecker Delta*

Prove that $\delta_{ij}\delta_{jk} = \delta_{ik}$

Answer: using a range of $i, j, k \in (1, 2, 3)$ δ_{ij} or δ_{jk} can only be non null if $i = j$ or $j = k$. Then only combination of $i = k$ will yield a result because by definition $i = j = k$ in this case. For example:

```
# Create a Kronecker Delta function
def deltakron(i,j):
    if i==j:
        return 1
    else:
        return 0

# Initialize sum variables
sum = 0
sum2 = 0
```

```

# Compute using i-j-k
for i in range(3):
    for j in range(3):
        for k in range(3):
            sum += deltakron(i,j)*deltakron(j,k)

# Compute using i-k
for i in range(3):
    for k in range(3):
        sum2 += deltakron(i,k)

```

As expected, $sum = 3$ and $sum2 = 3$

Problem 2.4 *Cross product using Levi-Civita symbol ϵ_{ijk}*

Compute the cross product of two vectors $\mathbf{a} \times \mathbf{b}$ using the Levi-Civita symbol.

Answer: In reference to the worked example, see section 2.1.3. The numeric resolution gives:

```

from sympy import Eijk

# Initialize two 3x1 vectors
a = MatrixSymbol('a', 3, 1)
b = MatrixSymbol('b', 3, 1)

# Since MatrixSymbol doesnt support insertion, a list and a sum are initialized
c = []
# Loop over indices and sum
for i in range(3):
    sum = 0
    for j in range(3):
        for k in range(3):
            sum += Eijk(i,j,k)*a[j]*b[k]

    c.append(sum)

```

$$c = \begin{bmatrix} a_{1,0}b_{2,0} - a_{2,0}b_{1,0} \\ -a_{0,0}b_{2,0} + a_{2,0}b_{0,0} \\ a_{0,0}b_{1,0} - a_{1,0}b_{0,0} \end{bmatrix}$$

Problem 2.5 *Trace*

Compute the trace of A using tensor notation

Answer: The trace is computed as A_{ii} . Since i is repeated twice it is summed. Thus:


```

# Intialize a 3x3 matrix
A = MatrixSymbol('A',3,3)

# Initialize a sum to compute trace
sum = 0

for i in range(3):
    sum += A[i,i]

# Compute trace using numpy
sum2 = np.trace(A)

```

As expected, $sum = A[0,0] + A[1,1] + A[2,2]$ and $sum2 = A[0,0] + A[1,1] + A[2,2]$

2.2 Deformation gradient and motion

Section References

The references used in this section are:

- Belytschko [1]
 - 3.2.2-3.2.3
 - 3.2.6
 - 3.7.1
- Continuum Mechanics website [2]
 - Deformation Gradient
 - Polar Decomposition

2.2.1 Introduction

Continuum mechanics introduce the concepts of **material** and **spatial** definition. See fig. 2.1. The difference between material and spatial is the definition of what is observed:

This subsection has relied heavily on [2]. Most of the notes are a literal copy of the reference. Especially for the explanation of the rotation and stretch tensor

Key Concept 2.2.1: Material vs Spatial description

- Material $\rightarrow$ Lagrangian: Following a person in a crowd. Typical in solid mechanics (FEA).
- Spatial $\rightarrow$ Eulerian: Fixing a point in a crowd and seeing who passes through. Typical in fluid mechanics (CFD).

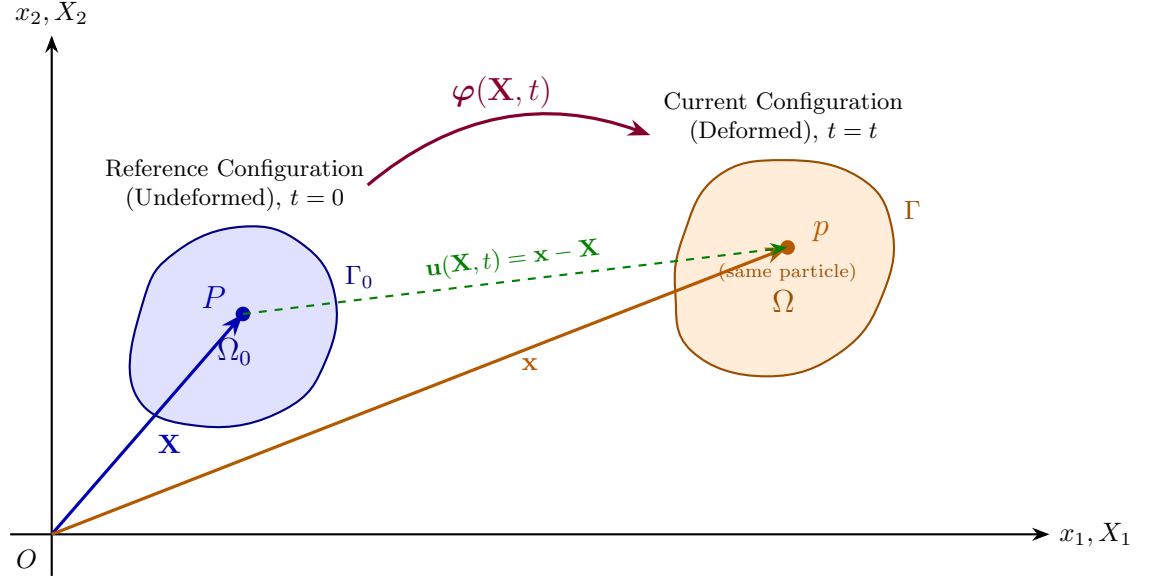


Figure 2.1: Material and spatial definition of continuum

The configuration of a body is defined by the symbol Ω and its boundaries are represented by Γ . The subscript $_0$ indicates an initial state. To go from Ω_0 to Ω , the body goes from $\mathbf{X}$ to $\mathbf{x}$. Essentially $\mathbf{X}$ means which particle (where is it initially) and $\mathbf{x}$ means where is it now? The idea is the same for the definition of any point $\mathbf{P}$ and $\mathbf{p}$. Then, how $\mathbf{x}$ relates to $\mathbf{X}$ is defined by the deformation map $\varphi(\mathbf{X}, t)$ and its displacement $\mathbf{u} = \mathbf{x} - \mathbf{X}$. The deformation map can be seen as a tracker in the "crowd" example above: Given this initial point in space, what happened to whatever went through this point initially.

[H]

2.2.2 Deformation Gradient

The deformation gradient $\mathbf{F}$ is simply the derivative of each component of the deformed vector from the deformed configuration: $\mathbf{x}$ with respect to each component of the reference configuration $\mathbf{X}$:

$$F_{ij} = x_{i,j} = \frac{\partial x_i}{\partial X_j} = \begin{bmatrix} \frac{\partial x_1}{\partial X_1} & \frac{\partial x_1}{\partial X_2} & \frac{\partial x_1}{\partial X_3} \\ \frac{\partial x_2}{\partial X_1} & \frac{\partial x_2}{\partial X_2} & \frac{\partial x_2}{\partial X_3} \\ \frac{\partial x_3}{\partial X_1} & \frac{\partial x_3}{\partial X_2} & \frac{\partial x_3}{\partial X_3} \end{bmatrix} \quad (2.1)$$

Example 2.2.1: Rigid body motion

Given the following mapping:

$$\mathbf{u} = \begin{bmatrix} x = X + 5 \\ y = Y + 2 \end{bmatrix}$$

Find the deformation gradient.

Answer: Here, $\mathbf{F}$ is simply $\mathbf{I}$ because

$$\mathbf{F} = \frac{\partial x_i}{\partial X_j} = \begin{bmatrix} \frac{\partial(X+5)}{\partial X} & \frac{\partial(X+5)}{\partial Y} \\ \frac{\partial(Y+2)}{\partial X} & \frac{\partial(Y+2)}{\partial Y} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Thus, the body is not deformed hence the title *Rigid body*.

Example 2.2.2: Rotation

Given the following mapping:

$$\mathbf{u} = \begin{bmatrix} x = X \cos \theta - Y \sin \theta \\ y = X \sin \theta + Y \cos \theta \end{bmatrix}$$

Find the deformation gradient.

Answer: $\mathbf{F}$ is then:

$$\mathbf{F} = \frac{\partial x_i}{\partial X_j} = \begin{bmatrix} \frac{\partial(X \cos \theta - Y \sin \theta)}{\partial X} & \frac{\partial(X \cos \theta - Y \sin \theta)}{\partial Y} \\ \frac{\partial(X \sin \theta + Y \cos \theta)}{\partial X} & \frac{\partial(X \sin \theta + Y \cos \theta)}{\partial Y} \end{bmatrix}$$

$$\mathbf{F} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Key Concept 2.2.2: Deformation gradient and hourglassing

Sometimes, displaced bodies that undergo true deformation do not yield deformation when computed. This is the case with hourglassing, a numerical error in finite element when elements do not yield any deformation when in fact they deform. It is typical in dominant bending problems and when selected elements use reduced integration. While this is out of topic, it is an important concept and gives a good example of the calculation of deformation gradient. Example adapted and taken from [2].

The displacement function is given by:

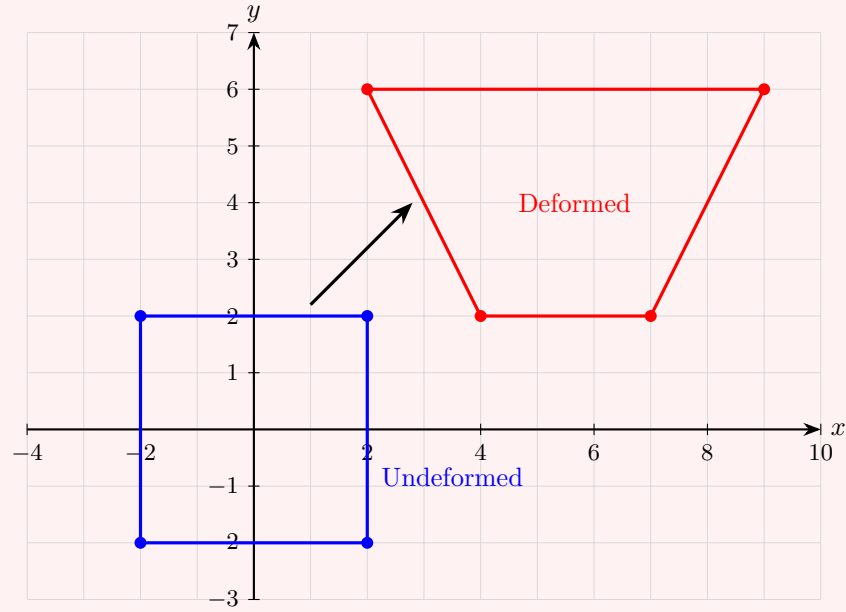
$$\mathbf{u} = \begin{bmatrix} X + \frac{1}{4}XY + 5 \\ Y + 4 \end{bmatrix}$$

The deformation gradient is then:

$$\mathbf{F} = \begin{bmatrix} 1 + \frac{1}{4}Y & \frac{1}{4}X \\ 0 & 1 \end{bmatrix}$$

at $(0,0)$, the initial position of the centre of the square, the deformation gradient is $\mathbf{I}$ since $X = 0$ and $Y = 0$, even though visually the body deforms:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$



Example of hourglassing and deformed configuration of a body

The determinant of the deformation gradient is called the jacobian determinant:

$$\mathbf{J} = \det \mathbf{F} \quad (2.2)$$

It represents the volume ratio of a body:

$$\begin{cases} J = 1 & \text{Incompressible} \\ J > 1 & \text{Expanding volume} \\ J < 1 & \text{Compressed volume} \end{cases}$$

It must obey:

$$\mathbf{J} > 0 \quad (2.3)$$

Because $\mathbf{F}$:

1. Must have an inverse
2. Would mean negative volume

2.2.3 Polar Decomposition

The subject of rotation has been approached in a previous example, but this subject is rather important in solid mechanics. Indeed, each deformed body can be explained in parts by translation (stretch) and rotation. In large displacement solid mechanics, rotation do not generate strain. Thus, the deformation gradient can be obtained by:

1. Rotating first then stretch, which gives:

$$\mathbf{F} = \mathbf{R} \cdot \mathbf{U} \quad (2.4)$$

2. Stretching first then rotate, which gives:

$$\mathbf{F} = \mathbf{V} \cdot \mathbf{R} \quad (2.5)$$

Here $\mathbf{U}$ and $\mathbf{V}$ are the right stretch tensor and left stretch tensor respectively. $\mathbf{R}$ is the rotation matrix. From eq. (2.4) and eq. (2.5) we can confidently say that:

$$\mathbf{V} \cdot \mathbf{R} = \mathbf{R} \cdot \mathbf{U} \quad (2.6)$$

Key Concept 2.2.3: Interesting properties of the rotation matrix R

The followings are two interesting properties of the rotation matrix:

$$\det R = 1 \quad (2.7)$$

$$R^{-1} = R^T \rightarrow \text{Orthogonal property} \quad (2.8)$$

by multiplying both sides by $\mathbf{R}^T$:

$$\mathbf{V} \cdot \mathbf{R} \cdot \mathbf{R}^T = \mathbf{R} \cdot \mathbf{U} \cdot \mathbf{R}^T \quad (2.9)$$

$$\mathbf{V} \cdot \mathbf{I} = \mathbf{R} \cdot \mathbf{U} \cdot \mathbf{R}^T \quad (2.10)$$

Thus,

$$\mathbf{V} = \mathbf{R} \cdot \mathbf{U} \cdot \mathbf{R}^T \quad (2.11)$$

and,

$$\mathbf{U} = \mathbf{R}^T \cdot \mathbf{V} \cdot \mathbf{R} \quad (2.12)$$

Knowing the interesting properties above, the partitioning of stretch and rotation can be determined using:

$$\mathbf{F}^T \cdot \mathbf{F} = (\mathbf{R} \cdot \mathbf{U})^T \cdot (\mathbf{R} \cdot \mathbf{U})^T = \mathbf{U}^T \cdot \mathbf{R}^T \cdot \mathbf{R} \cdot \mathbf{U} \quad (2.13)$$

The same goes with the left stretch tensor ($\mathbf{V}$) thus

$$\mathbf{F}^T \cdot \mathbf{F} = \mathbf{U}^T \cdot \mathbf{U} = \mathbf{V}^T \cdot \mathbf{V} \quad (2.14)$$

Example 2.2.3: Stretching and Rotation of a body

Given the following coordinate mapping:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1.30X - 0.375Y \\ 0.750X + 0.650Y \end{bmatrix}$$

Find the deformation gradient and partition the rotation and stretch tensor.

Answer: Use the right stretch tensor $\mathbf{U}$ to find $\mathbf{R}$. First, the deformation gradient:

$$\mathbf{F} = \frac{x_i}{X_j} = \begin{bmatrix} \frac{\partial(1.30X - 0.375Y)}{\partial(0.750X + 0.650Y)} & \frac{\partial(1.30X - 0.375Y)}{\partial(0.750X + 0.650Y)} \\ \frac{\partial(0.750X + 0.650Y)}{\partial X} & \frac{\partial(0.750X + 0.650Y)}{\partial Y} \end{bmatrix}$$

$$\mathbf{F} = \begin{bmatrix} 1.300 & -0.375 \\ 0.750 & 0.650 \end{bmatrix}$$

Hence, $\mathbf{F}^T \cdot \mathbf{F} = \mathbf{U}^T \cdot \mathbf{U}$. Thus we find $\mathbf{U}$ from taking the square root of $\mathbf{F}^T \cdot \mathbf{F}$:

$$\mathbf{U} = \sqrt{\mathbf{U}^T \cdot \mathbf{U}}$$

$$\mathbf{U} = \sqrt{\begin{bmatrix} 2.500 & 0.000 \\ 0.000 & 0.536 \end{bmatrix}} = \begin{bmatrix} 1.500 & 0.000 \\ 0.000 & 0.75 \end{bmatrix}$$

Then we multiply $\mathbf{F}$ by $\mathbf{U}^{-1}$ to obtain $\mathbf{R}$

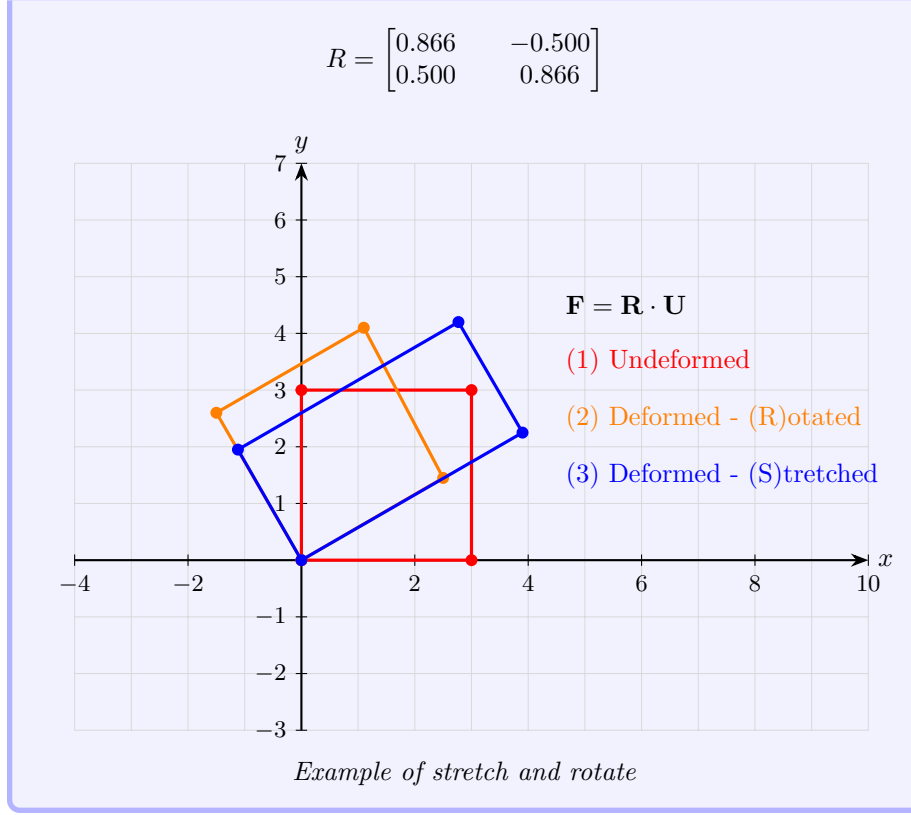
$$\mathbf{F} \cdot \mathbf{U}^{-1} = \mathbf{R} \cdot \mathbf{U} \cdot \mathbf{U}^{-1} = \mathbf{R}$$

where

$$\mathbf{U}^{-1} = \begin{bmatrix} 0.667 & 0.000 \\ 0.000 & 1.333 \end{bmatrix}$$

thus,

$$\mathbf{R} = \mathbf{F} \cdot \mathbf{U}^{-1} = \begin{bmatrix} 1.300 & -0.375 \\ 0.750 & 0.650 \end{bmatrix} \begin{bmatrix} 0.667 & 0.000 \\ 0.000 & 1.333 \end{bmatrix}$$



Note that other methods to find $\mathbf{R}$ exist, to list a few:

1. Using the left stretch tensor $\mathbf{F} = \mathbf{V}\mathbf{R}$
2. Iterative procedure:

$$A_{n+1} = \frac{1}{2}(A_n + A_n^{-T})$$

where $\mathbf{A}_n$ is replaced by $\mathbf{F}$. However, the transpose and inverse of $\mathbf{F}$ have to be computed to find F_n^{-T} . After a few iterations, this yields $\mathbf{R}$. Then, $\mathbf{V}$ or $\mathbf{U}$ can be found by using the extensions of eq. (2.4) and eq. (2.5), remember $\mathbf{R}^T = \mathbf{R}^{-1}$:

$$\mathbf{R}^{-1}\mathbf{F} = \mathbf{R}^{-1}\mathbf{R}\mathbf{U}$$

and

$$\mathbf{F}\mathbf{R}^{-1} = \mathbf{V}\mathbf{R}\mathbf{R}^{-1}$$

yields:

$$\mathbf{U} = \mathbf{R}^T\mathbf{F} \quad (2.15)$$

and:

$$\mathbf{V} = \mathbf{F}\mathbf{R}^T \quad (2.16)$$

2.2.4 Problem set

Problem 2.6 *Deformation gradient interpretation*

Given the deformation map φ :

$$\varphi = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 1.2X \\ 0.8Y \\ Z \end{bmatrix}$$

Find $\mathbf{F}$ and $\mathbf{J}$, interpret.

Answer: F is defined as:

```
from sympy import Matrix, symbols
X, Y, Z = symbols('X Y Z')
ref_coords = Matrix([X, Y, Z])

# Define the deformation map
phi = Matrix(
    [1.2*X,
     0.8*Y,
     1.0*Z])

# Calculate the deformation gradient (F)
F = phi.jacobian(ref_coords)

# Calculate the Jacobian (det F)
J = F.det()
```

Finally,

$$F = \begin{bmatrix} 1.2 & 0 & 0 \\ 0 & 0.8 & 0 \\ 0 & 0 & 1.0 \end{bmatrix}$$

$$\det F = J = 0.9600000000000000$$

Since $\mathbf{J} < 1$, the body is undergoing compression since the stretch in $\mathbf{X}$ is larger than the stretch in the other directions.

Problem 2.7 *Deformation gradient and rotation*

Find $\mathbf{F}$ for a rotation of 30° . Verify that $\det \mathbf{F} = 1$ and $\mathbf{F}^T \mathbf{F} = \mathbf{I}$

Answer:: The rotation matrix $\mathbf{R}$ in 2D is defined as:

$$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

We replace $\theta = 30^\circ$, and compute $\mathbf{F}$.


```
import numpy as np

# Define the angle
theta = np.deg2rad(30)

# Define the deformation gradient
F = np.array([
    [np.cos(theta), -np.sin(theta)],
    [np.sin(theta), np.cos(theta)]
])

# Verify that F^t * F = I
verif1 = np.transpose(F) * F

# Verify that det(F) = 1
verif2 = np.linalg.det(F)
```

We can then verify that:

$$\mathbf{F}^T \mathbf{F} = \mathbf{I} = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix}$$

and

$$\mathbf{J} = \det \mathbf{F} = 1.0$$

Since rotation is considered rigid body motion, the volume does not undergo deformation which makes since since the jacobian is unity.

Problem 2.8 *Deformation gradient and polar decomposition*

Given

$$\mathbf{F} = \begin{bmatrix} 1.3 & -0.375 \\ 0.75 & 0.65 \end{bmatrix}$$

Compute the polar decomposition (Find $\mathbf{C}$, $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{R}$).

Answer:: From eq. (2.14) We take the square root of $\mathbf{F}^T \mathbf{F}$ to find $\mathbf{U}$. Then we compute the inverse of $\mathbf{U}$ to find $\mathbf{R}$, see eq. (2.4):

```
import numpy as np
import scipy as sp

# Define the deformation gradient
F = np.array([
    [1.3, -0.375],
    [0.75, 0.65]
```

1)

```
# Compute C = F^T F => right Cauchy-Green deformation tensor
C = np.matmul(np.transpose(F), F)

# Compute U = sqrt(C)
U = np.sqrt(C)

# Compute R = F U^-1
R = np.matmul(F, np.linalg.inv(U))

# Compute V = F R^-1
V = np.matmul(F, np.linalg.inv(R))

# Validation using sp.linalg.polar
R2, U2 = sp.linalg.polar(F)

# Find R using iterative approach

Ak = F

Ak1 = 0.5 * (Ak + np.transpose(np.linalg.inv(Ak)))
Ak2 = 0.5 * (Ak1 + np.transpose(np.linalg.inv(Ak1)))
Ak3 = 0.5 * (Ak2 + np.transpose(np.linalg.inv(Ak2)))
```

Giving,

$$C = \begin{bmatrix} 2.25 & 0.0 \\ 0.0 & 0.56 \end{bmatrix}$$

$$U = \begin{bmatrix} 1.5 & 0.0 \\ 0.0 & 0.75 \end{bmatrix} = U2 = \begin{bmatrix} 1.5 & 0.0 \\ 0.0 & 0.75 \end{bmatrix}$$

$$V = \begin{bmatrix} 1.31 & 0.32 \\ 0.32 & 0.94 \end{bmatrix}$$

$$R = \begin{bmatrix} 0.87 & -0.5 \\ 0.5 & 0.87 \end{bmatrix} = R2 = \begin{bmatrix} 0.87 & -0.5 \\ 0.5 & 0.87 \end{bmatrix} \quad Ak = \begin{bmatrix} 1.3 & -0.38 \\ 0.75 & 0.65 \end{bmatrix}$$

Then, the validation of the iterative approach to find $\mathbf{R}$ using 3 steps

$$A_{k(0)} = \begin{bmatrix} 1.3 & -0.38 \\ 0.75 & 0.65 \end{bmatrix}$$

$$A_{k(1)} = \begin{bmatrix} 0.94 & -0.52 \\ 0.54 & 0.9 \end{bmatrix}, \epsilon = 5.0877e - 01$$

$$A_{k(2)} = \begin{bmatrix} 0.87 & -0.5 \\ 0.5 & 0.87 \end{bmatrix}, \epsilon = 9.0053e - 02$$

$$A_{k(3)} = \begin{bmatrix} 0.87 & -0.5 \\ 0.5 & 0.87 \end{bmatrix}, \epsilon = 3.3216e - 03$$

The polar decomposition using *sp.linalg.polar* gives the same results as the manual decomposition using *numpy*. Also, the iterative procedure gives quick results without too much effort to find $\mathbf{R}$

Problem 2.9 *Volume change and J*

Compute the volume change given

$$\mathbf{F} = \begin{bmatrix} 1.5 & 0 & 0 \\ 0 & 1.2 & 0 \\ 0 & 0 & 1.8 \end{bmatrix}$$

Answer:: Compute the jacobian (J):

```
import numpy as np

# Define the deformation gradient
F = np.array([
    [1.5, 0, 0],
    [0, 1.2, 0],
    [0, 0, 1.8]
])

# Compute J=det(F)
J = np.linalg.det(F)
```

Since $J = 3.24 > 1$. The body becomes 3.24 larger, thus the volume is 3.24 times bigger.

2.3 Motion

As described earlier, the displacement of a body is defined by:

$$\mathbf{u} = \mathbf{x} - \mathbf{X} \quad (2.17)$$

and the map from $\mathbf{X}$ to $\mathbf{x}$ is:

$$\mathbf{x} = \varphi(\mathbf{X}, t) \quad (2.18)$$

The velocity $\mathbf{v}(\mathbf{X}, t)$ is then, from eq. (2.17) and eq. (2.18):

$$\mathbf{v}(\mathbf{X}, t) = \frac{\partial \varphi(\mathbf{X}, t)}{\partial t} = \frac{\partial \mathbf{u}(\mathbf{X}, t)}{\partial t} \equiv \dot{\mathbf{u}} \quad (2.19)$$

in the same fashion, the acceleration is:

$$\mathbf{a}(\mathbf{X}, t) = \frac{\partial^2 \varphi(\mathbf{X}, t)}{\partial t^2} = \frac{\partial^2 \mathbf{u}(\mathbf{X}, t)}{\partial t^2} = \frac{\partial \mathbf{v}(\mathbf{X}, t)}{\partial t} \equiv \ddot{\mathbf{u}} \quad (2.20)$$

eq. (2.19) and eq. (2.20) both relates to the spatial description of the body (point in the crowd). To define a quantity using the spatial description (person in the crowd), one has to define a material derivative using the chain rule.

$$\frac{Dv_i(\mathbf{x}, t)}{Dt} = \frac{\partial v_i(\mathbf{x}, t)}{\partial t} + \frac{\partial v_i(\mathbf{x}, t)}{\partial x_j} \frac{\partial \varphi_i(\mathbf{X}, t)}{\partial t} \quad (2.21)$$

The derivative of the deformation map φ with respect to time, in the second term on the right hand side yields:

$$\frac{\partial v_i(\mathbf{x}, t)}{\partial x_j} \frac{\partial \varphi_i(\mathbf{X}, t)}{\partial t} = v_j \quad (2.22)$$

Which finally give:

$$\frac{Dv_i(\mathbf{x}, t)}{Dt} = \frac{\partial v_i}{\partial t} + \frac{\partial v_i}{\partial x_j} v_j \quad (2.23)$$

Or in matrix-vector from:

$$\frac{D\mathbf{v}(\mathbf{x}, t)}{Dt} = \frac{\partial \mathbf{v}(\mathbf{x}, t)}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} \quad (2.24)$$

which gives, in a more generic form:

$$\underbrace{\frac{D(\cdot)}{Dt}}_{\text{Material Derivative}} = \underbrace{\frac{\partial(\cdot)}{\partial t}}_{\text{Spatial term}} + \underbrace{\mathbf{v} \cdot \nabla(\cdot)}_{\text{Convective/Adjective term}} \quad (2.25)$$

Example 2.3.1: Ball drop

The position of a falling object, with respect to time and initial height (Y), is given by:

$$\varphi(\mathbf{X}, t) = y = Y + v_0 t - \frac{1}{2} g t^2$$

With,

$$\mathbf{u} = \mathbf{x} - \mathbf{X}$$

The definition of $\mathbf{u}$ becomes:

$$\mathbf{u}(\mathbf{X}, t) = \varphi(\mathbf{X}, t) - \mathbf{X}$$

Since $\mathbf{X} = 0 = \text{constant}$, $\mathbf{u}$ becomes:

$$\mathbf{u}(\mathbf{X}, t) = \varphi(\mathbf{X}, t) = Y + v_0 t - \frac{1}{2} g t^2$$

Then the velocity is:

$$\mathbf{v}(\mathbf{X}, t) = \frac{\partial \mathbf{u}(\mathbf{X}, t)}{\partial t} = v_0 - gt$$

and the acceleration:

$$\mathbf{a}(\mathbf{X}, t) = \frac{\partial \mathbf{v}(\mathbf{X}, t)}{\partial t} = -g$$

Then what is the velocity field (velocity at any point and time)? From eq. (2.25), we find that:

$$\mathbf{v}(\mathbf{x}, t) = \frac{D\mathbf{u}(\mathbf{x}, t)}{Dt} = \frac{\partial \mathbf{u}(\mathbf{x}, t)}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{u}$$

WIP

Bibliography

- [1] Ted Belytschko, W. K. Liu, B. Moran, and Khalil I. Elkhodary. *Nonlinear finite elements for continua and structures*. Hoboken, New Jersey : Wiley, 2014.
- [2] Bob McGinty. Continuum mechanics.